

Heaps

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Heaps	1
1.1	What Is a Heap, Really?	1
1.2	Implement a Min Heap	3
1.3	Check if an Array Represents a Heap	6
1.4	Convert a Min Heap to a Max Heap	8
1.5	Kth Largest Element in an Array	11
1.6	Kth Smallest Element in an Array	13
1.7	Hand of Straights	15
1.8	Merge K Sorted Lists	18
1.9	Replace Elements by Rank in the Array	21
1.10	Task Scheduler	24
1.11	Design Twitter	27
1.12	Find Median from a Data Stream	30
1.13	Kth Largest Element in a Stream	33
1.14	Maximum Sum Combination	35
1.15	Minimum Cost to Connect Sticks	38
1.16	Top K Frequent Elements	40

Chapter 1

Heaps

A heap does not know the full order of its elements, and it does not need to. It only ever promises one thing: the element at the root is the best (smallest, or largest) of everything currently inside – and that single promise, maintained cheaply after every insertion and removal, is enough to power an entire family of “best/kth/top” problems.

1.1 What Is a Heap, Really?

A **heap** (also called a **priority queue** when discussed in terms of its interface rather than its array-based implementation) is a tree-shaped data structure, almost always stored compactly in a plain array, that maintains one structural invariant: in a **min-heap**, every node’s value is $\leq$ the values of its children; in a **max-heap**, every node’s value is $\geq$ the values of its children. This invariant is weaker than full sortedness – a heap does not know the relative order of two sibling subtrees – but it is exactly strong enough to guarantee that the root is always the minimum (or maximum) of the whole structure, and that this root can be removed and the invariant restored in $O(\log n)$ time.

The Three Operations That Power Everything in This Chapter

- *push(x)*: insert x at the next free array slot, then *sift up* – repeatedly swap it with its parent while it violates the heap invariant relative to that parent. Cost: $O(\log n)$.
- *pop()*: read the root (the answer), move the last element into the root’s slot, shrink the array by one, then *sift down* – repeatedly swap the new root with whichever child violates the invariant most, until the invariant holds again. Cost: $O(\log n)$.
- *top()*: read the root without removing it. Cost: $O(1)$.

Every problem in this chapter is, underneath, a creative use of just these three operations.

How to Recognise a Heap Problem from Its Wording

Recurring Heap Keywords

- **“Kth largest/smallest”** – maintain a heap of size k of the opposite polarity (a min-heap of size k for “ k th largest,” so the root is the worst of your current top- k , ready to be evicted the instant something better arrives).
- **“Top k frequent/most/best”** – the same size- k heap trick, ranked by a derived key (frequency, sum, distance) rather than the raw value.
- **“Merge k sorted ...”** – a heap holding one “current head” candidate from each of the k sources at a time, repeatedly popping the smallest and pushing that source’s next element.
- **“Running/streaming median, or any other running statistic that needs both**

a top half and a bottom half” – two heaps, one min-heap and one max-heap, used together as a self-balancing pair.

- **“Schedule/group/connect with a cost or cooldown that always prefers the largest (or smallest) remaining item first”** – a single heap used as a perpetually-resorted greedy priority queue, replacing what would otherwise require re-sorting after every step.

Whenever brute force would require repeatedly finding “the current best/worst remaining element” after the underlying collection keeps changing, a heap is almost always the efficient replacement for “sort again from scratch every time.”

Each section in this chapter follows the same structure as the rest of this book: the problem statement and its keywords; the reasoning that identifies which heap (or pair of heaps) to use and why; the algorithm in words and in formal pseudocode; a hand-traced dry run; complete compilable C++ code (using `std::priority_queue` throughout, since that is what production C++ code – and nearly every interview answer – actually uses); and a time/space complexity analysis.

1.2 Implement a Min Heap

Problem Statement. Implement a min-heap supporting three operations: *push*(*x*) (insert a new value), *pop*() (remove and return the smallest value), and *top*() (peek at the smallest value without removing it), each in $O(\log n)$ time for push/pop and $O(1)$ for top.

Pattern Recognition

This section has no “keyword” in the usual sense – it is the mechanical foundation every other section in this chapter builds on. The skill being tested is not pattern recognition but the array-indexing arithmetic that lets a heap live inside a flat array without ever needing explicit left/right/parent pointers.

Understanding the array-as-tree representation

Store the heap’s elements in a plain array *heap*[0..*n* − 1], interpreted as a complete binary tree read level by level, left to right. For a node at index *i* (0-indexed):

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor, \quad \text{left}(i) = 2i+1, \quad \text{right}(i) = 2i+2.$$

This works because a complete binary tree (every level full except possibly the last, which fills left to right with no gaps) has exactly this relationship between a node’s position in level order and its children’s positions – no pointers are needed, only arithmetic.

Sift Up (used by push)

Append the new value at index *n* (the next free slot). While this node’s value is smaller than its parent’s value, swap them and continue checking from the parent’s old position. This restores the min-heap invariant locally along the single path from the new leaf up to the root, which is always sufficient: every other path in the tree was already valid before the insertion and is untouched by these swaps.

Sift Down (used by pop)

Save the root’s value (this is the return value). Move the *last* element into the root’s slot and shrink the array by one. While this relocated value is larger than the smaller of its two children, swap it with that smaller child and continue checking from the new position. This restores the invariant locally along a single path from the root downward.

Why sifting along a single path is always sufficient

Both operations rely on the same structural fact: the min-heap invariant is a *local* condition (each node versus its own parent/children), and inserting or removing one element only disturbs the invariant along *one* root-to-leaf path – the path the new or relocated element travels. Every other node’s relationship to its own parent and children is completely unaffected, so checking and fixing violations one step at a time along that single path, stopping the instant no violation remains, is guaranteed to restore a fully valid heap. This is also exactly why each operation costs $O(\log n)$: a path from root to leaf in a complete binary tree with *n* nodes has length $O(\log n)$, since the tree’s height is $\lfloor \log_2 n \rfloor$.

Algorithm

Push(*x*):

1. Append x at index n ; let $i \leftarrow n$; increment n .
 2. While $i > 0$ and $\text{heap}[\text{parent}(i)] > \text{heap}[i]$: swap them; $i \leftarrow \text{parent}(i)$.
- Pop():**
1. Save $\text{result} \leftarrow \text{heap}[0]$.
 2. $\text{heap}[0] \leftarrow \text{heap}[n - 1]$; decrement n .
 3. $i \leftarrow 0$. While i has a child smaller than itself: let smallerChild be the smaller of its two children (only those that exist); swap $\text{heap}[i]$ with $\text{heap}[\text{smallerChild}]$; $i \leftarrow \text{smallerChild}$.
 4. Return result .

Algorithm 1 Min Heap: Push and Pop

```

1: procedure PUSH( $x$ )
2:   append  $x$ ;  $i \leftarrow n - 1$ 
3:   while  $i > 0$  and  $\text{heap}[(i - 1)/2] > \text{heap}[i]$  do
4:     swap  $\text{heap}[(i - 1)/2]$  and  $\text{heap}[i]$ 
5:      $i \leftarrow (i - 1)/2$ 
6:   end while
7: end procedure
8: procedure POP
9:    $\text{result} \leftarrow \text{heap}[0]$ 
10:   $\text{heap}[0] \leftarrow \text{heap}[n - 1]$ ; remove last slot
11:   $i \leftarrow 0$ 
12:  while  $i$  has a smaller child do
13:     $j \leftarrow$  index of the smaller existing child of  $i$ 
14:    if  $\text{heap}[i] \leq \text{heap}[j]$  then break
15:    end if
16:    swap  $\text{heap}[i]$  and  $\text{heap}[j]$ ;  $i \leftarrow j$ 
17:  end while
18:  return  $\text{result}$ 
19: end procedure

```

Dry run

Dry Run. Push 5, 3, 8, 1 in order, then pop once.

- Push 5: $\text{heap} = [5]$.
- Push 3: append, $\text{heap} = [5, 3]$. Compare 3 to parent 5: $3 < 5$, swap. $\text{Heap} = [3, 5]$.
- Push 8: append, $\text{heap} = [3, 5, 8]$. Parent of index 2 is index 0, value 3. $8 \geq 3$, no swap. $\text{Heap} = [3, 5, 8]$.
- Push 1: append, $\text{heap} = [3, 5, 8, 1]$. Parent of index 3 is index 1, value 5. $1 < 5$, swap: $\text{heap} = [3, 1, 8, 5]$. New index 1; parent is index 0, value 3. $1 < 3$, swap: $\text{heap} = [1, 3, 8, 5]$.
- Pop: $\text{result} = 1$. Move last element (5) to root: $\text{heap} = [5, 3, 8]$. At index 0 (5): children at indices 1, 2 are 3, 8; smaller is 3 at index 1. $5 > 3$, swap: $\text{heap} = [3, 5, 8]$. At index 1 (5): only possible child would be at index 3, out of range. Stop.

The pop returns 1 (correctly the minimum), and the heap array $[3, 5, 8]$ still satisfies the min-heap invariant.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class MinHeap {
5      vector<int> heap;
6
7      void siftUp(int i) {
8          while (i > 0 && heap[(i - 1) / 2] > heap[i]) {
9              swap(heap[(i - 1) / 2], heap[i]);
10             i = (i - 1) / 2;
11         }
12     }
13
14     void siftDown(int i) {
15         int n = heap.size();
16         while (true) {
17             int left = 2 * i + 1, right = 2 * i + 2, smallest = i;
18             if (left < n && heap[left] < heap[smallest]) smallest = left;
19             if (right < n && heap[right] < heap[smallest]) smallest = right;
20             if (smallest == i) break;
21             swap(heap[i], heap[smallest]);
22             i = smallest;
23         }
24     }
25
26 public:
27     void push(int x) {
28         heap.push_back(x);
29         siftUp(heap.size() - 1);
30     }
31
32     int pop() {
33         int result = heap[0];
34         heap[0] = heap.back();
35         heap.pop_back();
36         if (!heap.empty()) siftDown(0);
37         return result;
38     }
39
40     int top() { return heap[0]; }
41     bool empty() { return heap.empty(); }
42 };
43
44 int main() {
45     MinHeap h;
46     h.push(5); h.push(3); h.push(8); h.push(1);
47     cout << "Popped: " << h.pop() << endl;
48     cout << "New top: " << h.top() << endl;
49     return 0;
50 }

```

Complexity Analysis

Time Complexity. Both *push* and *pop* touch only one root-to-leaf path, of length $O(\log n)$, doing $O(1)$ work per level, giving

$$O(\log n) \text{ per push/pop,} \quad O(1) \text{ per top.}$$

Space Complexity. $O(n)$ to store the n elements; no extra auxiliary structures are needed beyond the array itself.

Key Takeaway

Every problem in the rest of this chapter uses `std::priority_queue` directly rather than reimplementing this class, but understanding sift-up and sift-down by hand here is what makes the complexity claims in every later section ($O(\log n)$ per heap operation) something you can justify rather than merely recite.

1.3 Check if an Array Represents a Heap

Problem Statement. Given an array $arr[0..n-1]$, determine whether it represents a valid **max-heap** when interpreted as a complete binary tree in level order (every parent at index i should be $\geq$ its children at $2i+1$ and $2i+2$).

Pattern Recognition

This is a direct test of the array-as-tree arithmetic from Section 1.2: the keyword is simply “*does this array, read as a complete binary tree, satisfy the heap invariant everywhere.*” There is no clever algorithm to discover – the entire exercise is checking the invariant at every internal node using the $left(i) = 2i+1$, $right(i) = 2i+2$ formulas directly.

Understanding the problem carefully

A leaf node (one with no children, i.e. $2i+1 \geq n$) trivially satisfies the heap property, since it has nothing to compare against. So we only need to check internal nodes – indices i with $2i+1 < n$ – and verify that $arr[i] \geq arr[2i+1]$ (when the left child exists) and $arr[i] \geq arr[2i+2]$ (when the right child exists). If every internal node passes both checks, the whole array is a valid max-heap, because the heap property is purely local: validity at every parent-child pair, checked independently, is both necessary and sufficient for the global invariant to hold (there is no broader ordering constraint a heap is required to satisfy beyond these local pairwise comparisons).

Why Checking Only Internal Nodes Suffices

The heap property is defined entirely in terms of parent-versus-children comparisons. A node with no children imposes no constraint of its own, so the full set of constraints to verify is exactly the set of (parent, child) pairs that exist in the array – checking every internal node against its existing children covers all of them exactly once.

Algorithm

1. For $i \leftarrow 0$ to $n-1$, while $2i+1 < n$ (i.e. i is internal):
 - (a) If $2i+1 < n$ and $arr[i] < arr[2i+1]$: return **false**.

(b) If $2i + 2 < n$ and $arr[i] < arr[2i + 2]$: return **false**.

2. If every internal node passes, return **true**.

Algorithm 2 Check Array Represents a Max-Heap

```

1: procedure ISHEAP(arr)
2:   for  $i \leftarrow 0$  to  $n - 1$  do
3:      $left \leftarrow 2i + 1$ ,  $right \leftarrow 2i + 2$ 
4:     if  $left < n$  and  $arr[i] < arr[left]$  then
5:       return false
6:     end if
7:     if  $right < n$  and  $arr[i] < arr[right]$  then
8:       return false
9:     end if
10:  end for
11:  return true
12: end procedure

```

Dry run

Dry Run. Take $arr = [90, 15, 10, 7, 12, 2, 7, 3]$.

- $i = 0$ (value 90): children at 1, 2 are 15, 10. $90 \geq 15$ and $90 \geq 10$. OK.
- $i = 1$ (value 15): children at 3, 4 are 7, 12. $15 \geq 7$ and $15 \geq 12$. OK.
- $i = 2$ (value 10): children at 5, 6 are 2, 7. $10 \geq 2$ and $10 \geq 7$. OK.
- $i = 3$ (value 7): child at 7 is 3 (no right child, index 8 out of range). $7 \geq 3$. OK.
- $i = 4, 5, 6, 7$: no children ($2i + 1 \geq 8$). Trivially OK.

Every internal node passes, so the array is a valid max-heap.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isHeap(vector<int>& arr) {
5     int n = arr.size();
6     for (int i = 0; i < n; i++) {
7         int left = 2 * i + 1, right = 2 * i + 2;
8         if (left < n && arr[i] < arr[left]) return false;
9         if (right < n && arr[i] < arr[right]) return false;
10    }
11    return true;
12 }
13
14 int main() {
15     vector<int> arr = {90, 15, 10, 7, 12, 2, 7, 3};
16     cout << (isHeap(arr) ? "true" : "false") << endl;
17     return 0;
18 }

```

Complexity Analysis

Time Complexity. Each index is visited once with constant work, giving

$$O(n).$$

Space Complexity. $O(1)$ auxiliary space.

Key Takeaway

The heap property is purely local (parent versus its own children), never global – a heap is not required to be sorted in any stronger sense. This is precisely why heap operations are $O(\log n)$ rather than $O(n \log n)$: restoring a single local violation after an insertion or removal never requires re-examining the whole structure.

1.4 Convert a Min Heap to a Max Heap

Problem Statement. Given an array that currently satisfies the min-heap property, rearrange it in place so that it satisfies the max-heap property instead.

Pattern Recognition

The keyword here is “*convert*,” not “*rebuild from a sorted list*” – the existing min-heap order is, in fact, almost entirely irrelevant to solving this efficiently; the fastest approach simply **re-heapifies the array from scratch as a max-heap**, ignoring its current (min-heap) structure. This problem is the natural place to introduce the classic $O(n)$ **build-heap** algorithm, which is faster than the $O(n \log n)$ you would get by popping every element from the min-heap and pushing it into a fresh max-heap one at a time.

Understanding the problem carefully

The simplest correct approach: treat the existing array’s *values* as an unordered list of n numbers (discard the fact that they currently satisfy a min-heap arrangement – it buys us nothing for the new task), and re-heapify in place into a max-heap. The key efficiency insight is that *any* array can be turned into a valid heap in $O(n)$ time, not $O(n \log n)$, by sifting down starting from the **last internal node** and working backward to the root.

Why Starting from the Last Internal Node Works

A leaf node trivially satisfies the heap property on its own. If we process internal nodes in *reverse* level order (bottom-most, right-most internal node first, ending at the root), then by the time we sift-down a given node i , both of its children’s subtrees have *already* been fixed into valid max-heaps (since they were processed earlier, being lower in the tree). Sifting node i down then only needs to resolve a violation between i and the roots of two already-valid subtrees – exactly the situation sift-down is designed to fix – so a single bottom-up pass, sifting each internal node down once, is enough to turn the entire array into a valid heap.

Why this build-heap process is $O(n)$, not $O(n \log n)$

Each sift-down costs time proportional to the *height of the subtree rooted at that node*, not the height of the whole tree. Most nodes are near the bottom of the tree, where subtree heights are

tiny; only the handful of nodes near the root have a large subtree height. Formally, the total work is

$$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \cdot h = O(n),$$

since the number of nodes at height h from the bottom is roughly $n/2^{h+1}$, and this geometric-times-linear sum converges to $O(n)$ rather than the naive (and incorrect) $O(n \log n)$ upper bound you would get by assuming every node costs the full tree height.

Algorithm

1. The last internal node (one with at least one child) is at index $\lfloor n/2 \rfloor - 1$.
2. For i from $\lfloor n/2 \rfloor - 1$ down to 0: sift-down node i , using max-heap comparisons (swap with the *larger* child whenever a child exceeds the current node).
3. The array now satisfies the max-heap property.

Algorithm 3 Build a Max-Heap In Place (Heapify)

```

1: procedure BUILDMAXHEAP( $arr$ )
2:   for  $i \leftarrow \lfloor n/2 \rfloor - 1$  down to 0 do
3:     SIFTDOWNMAX( $arr, i, n$ )
4:   end for
5: end procedure
6: procedure SIFTDOWNMAX( $arr, i, n$ )
7:   while true do
8:      $largest \leftarrow i, left \leftarrow 2i + 1, right \leftarrow 2i + 2$ 
9:     if  $left < n$  and  $arr[left] > arr[largest]$  then
10:       $largest \leftarrow left$ 
11:     end if
12:     if  $right < n$  and  $arr[right] > arr[largest]$  then
13:       $largest \leftarrow right$ 
14:     end if
15:     if  $largest = i$  then break
16:     end if
17:     swap  $arr[i]$  and  $arr[largest]$ ;  $i \leftarrow largest$ 
18:   end while
19: end procedure

```

Dry run

Dry Run. Take the min-heap $arr = [1, 3, 5, 4, 6, 7, 8]$ ($n = 7$).

Last internal node: $\lfloor 7/2 \rfloor - 1 = 2$.

- $i = 2$ (value 5): children at 5, 6 are 7, 8. Largest child is 8 at index 6. $8 > 5$: swap. $arr = [1, 3, 8, 4, 6, 7, 5]$. Index 6 is a leaf; stop.
- $i = 1$ (value 3): children at 3, 4 are 4, 6. Largest is 6 at index 4. $6 > 3$: swap. $arr = [1, 6, 8, 4, 3, 7, 5]$. Index 4 is a leaf; stop.
- $i = 0$ (value 1): children at 1, 2 are 6, 8. Largest is 8 at index 2. $8 > 1$: swap. $arr = [8, 6, 1, 4, 3, 7, 5]$. Now at index 2 (value 1): children at 5, 6 are 7, 5. Largest is 7 at index 5. $7 > 1$: swap. $arr = [8, 6, 7, 4, 3, 1, 5]$. Index 5 is a leaf; stop.

The final array [8, 6, 7, 4, 3, 1, 5] satisfies the max-heap property (the root 8 is the maximum, and every parent exceeds its children).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void siftDownMax(vector<int>& arr, int i, int n) {
5     while (true) {
6         int largest = i, left = 2 * i + 1, right = 2 * i + 2;
7         if (left < n && arr[left] > arr[largest]) largest = left;
8         if (right < n && arr[right] > arr[largest]) largest = right;
9         if (largest == i) break;
10        swap(arr[i], arr[largest]);
11        i = largest;
12    }
13 }
14
15 void minHeapToMaxHeap(vector<int>& arr) {
16     int n = arr.size();
17     for (int i = n / 2 - 1; i >= 0; i--) {
18         siftDownMax(arr, i, n);
19     }
20 }
21
22 int main() {
23     vector<int> arr = {1, 3, 5, 4, 6, 7, 8}; // valid min-heap
24     minHeapToMaxHeap(arr);
25
26     cout << "Max-heap: ";
27     for (int x : arr) cout << x << " ";
28     cout << endl;
29     return 0;
30 }

```

Complexity Analysis

Time Complexity. The build-heap process visits $O(n/2)$ internal nodes with total sift-down work summing to

$$O(n)$$

across the whole array (not $O(n \log n)$), by the height-weighted geometric sum argument above. **Space Complexity.** $O(1)$ auxiliary space (the conversion is done in place).

Key Takeaway

Re-heapifying an entire array from scratch is $O(n)$, not $O(n \log n)$ – a fact that is easy to misremember because a *single* sift-down is $O(\log n)$, tempting you to multiply by n insertions. The $O(n)$ bound only holds when you build the whole heap at once via bottom-up sift-down, not when you insert n elements one at a time via repeated sift-up. `STD::MAKE_HEAP` in C++ uses exactly this $O(n)$ bottom-up technique internally.

1.5 Kth Largest Element in an Array

Problem Statement. Given an array *nums* of n integers and an integer k , find the k -th largest element in the array (the element that would be at index $n - k$ if the array were sorted in increasing order). Note this is the k -th largest *value*, counting duplicates by position, not the k -th distinct value.

Pattern Recognition

“*Kth largest*” is the single most direct trigger for the size-bounded heap technique introduced in Section 1.1’s keyword list. The key (and slightly counter-intuitive) design choice: to find the k -th **largest** element efficiently, maintain a **min**-heap, not a max-heap – the min-heap’s root is the *worst* element among your current best- k candidates, which is exactly the one you want instant access to evict.

Understanding the problem carefully

Sorting the whole array and reading off index $n - k$ costs $O(n \log n)$ and uses the full order of all n elements, even though we only care about one specific rank. A heap lets us track only the k largest elements seen so far, discarding everything else immediately.

Why a Min-Heap, not a Max-Heap

Maintain a min-heap that never holds more than k elements – think of it as “the best k candidates for being among the k largest, with the *worst* of those k candidates sitting at the root, ready to be evicted.” Process the array one element at a time: push the new element, then if the heap’s size exceeds k , pop the root (the smallest among the current top- k candidates, which is therefore the easiest one to safely discard). After processing every element, the heap holds exactly the k largest elements of the whole array, and its root – the smallest of those k – is precisely the k -th largest element overall.

Why evicting the heap’s root is always a safe discard

At any point during the scan, the min-heap holds some k candidates for “currently in the top k .” When a new element arrives and the heap already has k elements, we must decide: is the new element large enough to belong in the top k , displacing the current weakest member? Pushing the new element first and then popping the minimum handles both cases uniformly without an explicit if-statement: if the new element is smaller than everything already in the heap, it becomes the new minimum and is immediately popped right back out, leaving the heap unchanged in effect; if the new element is larger than the current minimum, that old minimum (now provably not among the true top k , since at least k other elements, including the new one, are now known to be at least as large) is correctly evicted instead. Either way, the heap always ends each step holding the k largest elements seen *so far*, by induction.

Algorithm

1. Initialize an empty min-heap.
2. For each element x in *nums*:
 - (a) Push x onto the heap.
 - (b) If the heap’s size exceeds k : pop the minimum.
3. Return the heap’s current minimum (its root).

Algorithm 4 Kth Largest Element

```

1: procedure KTHLARGEST(nums, k)
2:   heap  $\leftarrow$  empty min-heap
3:   for each x in nums do
4:     push x onto heap
5:     if  $|heap| > k$  then
6:       pop the minimum from heap
7:     end if
8:   end for
9:   return top of heap
10: end procedure

```

Dry run

Dry Run. Take $nums = [3, 2, 1, 5, 6, 4]$, $k = 2$.

- Push 3: $heap = \{3\}$. Size $1 \leq 2$.
- Push 2: $heap = \{2, 3\}$. Size $2 \leq 2$.
- Push 1: $heap = \{1, 2, 3\}$. Size $3 > 2$: pop min (1). Heap = $\{2, 3\}$.
- Push 5: $heap = \{2, 3, 5\}$. Size $3 > 2$: pop min (2). Heap = $\{3, 5\}$.
- Push 6: $heap = \{3, 5, 6\}$. Size $3 > 2$: pop min (3). Heap = $\{5, 6\}$.
- Push 4: $heap = \{4, 5, 6\}$. Size $3 > 2$: pop min (4). Heap = $\{5, 6\}$.

The final heap holds $\{5, 6\}$, the two largest elements; its root (minimum of these two) is 5, the 2nd largest element overall. Sorting $[1, 2, 3, 4, 5, 6]$ confirms index $n - k = 4$ holds 5.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int kthLargest(vector<int>& nums, int k) {
5      priority_queue<int, vector<int>, greater<int>> minHeap; // min-heap
6
7      for (int x : nums) {
8          minHeap.push(x);
9          if ((int)minHeap.size() > k) {
10             minHeap.pop();
11         }
12     }
13     return minHeap.top();
14 }
15
16 int main() {
17     vector<int> nums = {3, 2, 1, 5, 6, 4};
18     cout << "2nd largest: " << kthLargest(nums, 2) << endl;
19     return 0;
20 }

```

Complexity Analysis

Time Complexity. Each of the n elements triggers one push, and at most one pop, each costing $O(\log k)$ since the heap never exceeds size k , giving an overall time complexity of

$$O(n \log k),$$

a clear improvement over $O(n \log n)$ when $k \ll n$. **Space Complexity.** $O(k)$, since the heap never holds more than k elements.

Key Takeaway

The “opposite polarity” rule is worth memorizing precisely because it feels backward at first: for *largest- k* problems, use a *min*-heap of size k (so the easiest-to-discard candidate is always at the root); for *smallest- k* problems (Section 1.6), use a *max*-heap of size k . The heap’s root is always the *boundary* element of your current best- k set – the one you’re most willing to give up.

1.6 Kth Smallest Element in an Array

Problem Statement. Given an array *nums* of n integers and an integer k , find the k -th smallest element in the array.

Pattern Recognition

The mirror image of Section 1.5: “*Kth smallest*” calls for the *opposite*-polarity heap, a **max**-heap of size k . The root of a size- k max-heap is the *largest* of the current best- k -smallest candidates – exactly the one to discard the instant a smaller element arrives.

Understanding the problem carefully

By the same reasoning as Section 1.5, applied with every comparison flipped: maintain a max-heap that never holds more than k elements. Push each new element, then if the heap exceeds size k , pop the maximum (the current weakest member of the “smallest k so far” set). After processing the whole array, the heap holds the k smallest elements, and its root – the largest of those k – is the k -th smallest element overall.

Reuse, with Every Comparison Flipped

Apply Algorithm 1.5’s KTHLARGEST procedure verbatim, replacing the min-heap with a max-heap and “pop the minimum” with “pop the maximum.”

Algorithm

1. Initialize an empty max-heap.
2. For each element x in *nums*:
 - (a) Push x onto the heap.
 - (b) If the heap’s size exceeds k : pop the maximum.
3. Return the heap’s current maximum (its root).

Algorithm 5 Kth Smallest Element

```

1: procedure KTHSMALLEST(nums, k)
2:   heap  $\leftarrow$  empty max-heap
3:   for each x in nums do
4:     push x onto heap
5:     if |heap| > k then
6:       pop the maximum from heap
7:     end if
8:   end for
9:   return top of heap
10: end procedure

```

Dry run

Dry Run. Take $nums = [7, 10, 4, 3, 20, 15]$, $k = 3$.

- Push 7: $heap = \{7\}$.
- Push 10: $heap = \{7, 10\}$.
- Push 4: $heap = \{4, 7, 10\}$. Size $3 \leq 3$.
- Push 3: $heap = \{3, 4, 7, 10\}$. Size $4 > 3$: pop max (10). Heap = $\{3, 4, 7\}$.
- Push 20: $heap = \{3, 4, 7, 20\}$. Size $4 > 3$: pop max (20). Heap = $\{3, 4, 7\}$.
- Push 15: $heap = \{3, 4, 7, 15\}$. Size $4 > 3$: pop max (15). Heap = $\{3, 4, 7\}$.

The final heap holds $\{3, 4, 7\}$, the three smallest elements; its root (maximum of these three) is 7, the 3rd smallest element overall. Sorting $[3, 4, 7, 10, 15, 20]$ confirms index $k - 1 = 2$ holds 7.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int kthSmallest(vector<int>& nums, int k) {
5     priority_queue<int> maxHeap; // max-heap (default in C++)
6
7     for (int x : nums) {
8         maxHeap.push(x);
9         if ((int)maxHeap.size() > k) {
10             maxHeap.pop();
11         }
12     }
13     return maxHeap.top();
14 }
15
16 int main() {
17     vector<int> nums = {7, 10, 4, 3, 20, 15};
18     cout << "3rd smallest: " << kthSmallest(nums, 3) << endl;
19     return 0;
20 }

```

Complexity Analysis

Time Complexity.

$$O(n \log k),$$

identical in structure to Section 1.5. **Space Complexity.** $O(k)$.

Key Takeaway

Once Kth Largest is internalized, Kth Smallest costs nothing extra to derive – it is the same algorithm with every comparison polarity reversed. The general rule: a size- k heap of the *opposite* polarity to what you are looking for always keeps the right boundary element at the root.

1.7 Hand of Straights

Problem Statement. We are given a hand of n cards, $hand[0..n-1]$ (integers, possibly with repeats), and an integer $groupSize$. Determine whether the hand can be rearranged into groups, each of exactly $groupSize$ cards with **consecutive** values (e.g. $[3, 4, 5]$ is a valid group of size 3; $[3, 4, 6]$ is not). Return **true** or **false**.

Pattern Recognition

“Always form the next group starting from the smallest remaining card” is the greedy insight, and “repeatedly find the current smallest remaining distinct value” is exactly what a min-heap (over distinct values) is built for. Whenever a problem requires repeatedly peeling off the globally smallest (or largest) remaining item from a *changing* collection – not just once, but again and again as the collection shrinks – that repeated “smallest of what’s left” query is the heap keyword to watch for, distinguishing this from a single-pass greedy that only needs one sorted scan.

Understanding the problem carefully

First, if n is not divisible by $groupSize$, the answer is immediately **false** – the cards cannot be partitioned evenly. Otherwise, count the occurrences of each distinct card value (a hash map), and observe a key greedy fact: whichever card value is currently the **smallest** among all cards not yet placed into a group *must* be the starting card of some group – it cannot be the second or later card of any group, since no smaller card remains to precede it. So we are forced to start a new group at the current minimum, every time.

Greedy Choice, Powered by a Min-Heap

Maintain a min-heap of the *distinct* card values currently present in the hand (with their counts tracked separately in a hash map). Repeatedly: peek the smallest remaining value v ; this must start a new group. Check that $v, v+1, \dots, v+groupSize-1$ are all available (count > 0 in the hash map); if any is missing, return **false**. Otherwise, decrement the count of each of these $groupSize$ values; whenever a value’s count reaches 0, remove it from consideration (pop it from the heap, or simply leave it and skip over exhausted values when they resurface at the top). Repeat until all cards are consumed.

Why always starting at the current minimum is correct

This is another exchange-argument greedy: suppose, for contradiction, that some valid grouping does *not* use the smallest remaining card v as the start of a group, but instead places it later in

some group starting at $v' < v$ – but v is the minimum remaining, so no $v' < v$ can exist among the remaining cards. Therefore v cannot appear anywhere except as the start of a group (it cannot be the second card of a group starting at $v - 1$, because $v - 1$ is not present among the remaining cards by assumption that v is the minimum). So every valid grouping is forced to start a group at v , and the greedy choice has no alternative to consider in the first place – it is not just a good choice, it is the *only* choice consistent with feasibility.

Algorithm

1. If $n \bmod \text{groupSize} \neq 0$: return **false**.
2. Build a frequency map *count* of card values; build a min-heap of the distinct values present.
3. While the heap is not empty:
 - (a) Pop values from the top of the heap whose count has reached 0 (already fully used) and discard them.
 - (b) Peek the new minimum v (if the heap is now empty, stop – all cards are used).
 - (c) For $j \leftarrow 0$ to $\text{groupSize} - 1$: if $\text{count}[v + j] = 0$, return **false**; otherwise decrement $\text{count}[v + j]$.
4. Return **true**.

Algorithm 6 Hand of Straights

```

1: procedure ISNSTRAIGHTHAND(hand, groupSize)
2:   if  $n \bmod \text{groupSize} \neq 0$  then return false
3:   end if
4:   build frequency map count; build min-heap of distinct values
5:   while heap is not empty do
6:     while heap not empty and  $\text{count}[\text{top}] = 0$  do
7:       pop heap
8:     end while
9:     if heap is empty then break
10:    end if
11:     $v \leftarrow \text{top of heap}$ 
12:    for  $j \leftarrow 0$  to  $\text{groupSize} - 1$  do
13:      if  $\text{count}[v + j] = 0$  then return false
14:      end if
15:       $\text{count}[v + j] \leftarrow \text{count}[v + j] - 1$ 
16:    end for
17:  end while
18:  return true
19: end procedure

```

Dry run

Dry Run. Take $\text{hand} = [1, 2, 3, 6, 2, 3, 4, 7, 8]$, $\text{groupSize} = 3$.

$n = 9$, $9 \bmod 3 = 0$. Counts: 1:1, 2:2, 3:2, 4:1, 6:1, 7:1, 8:1. Min-heap of distinct values: $\{1, 2, 3, 4, 6, 7, 8\}$.

- Min is 1. Group $[1, 2, 3]$: all counts > 0 . Decrement: 1:0, 2:1, 3:1.

- Top of heap is 1, but $count[1] = 0$ now; pop it. New min is 2. Group [2, 3, 4]: all counts > 0 . Decrement: 2:0, 3:0, 4:0.
- Pop 2, 3, 4 as their counts reach 0 at the top. New min is 6. Group [6, 7, 8]: all counts > 0 (1 each). Decrement to 0 each.
- Heap empties out as exhausted values are popped.

All cards consumed without failure, so the answer is **true**: the hand splits into [1, 2, 3], [2, 3, 4], [6, 7, 8].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 bool isNStraightHand(vector<int>& hand, int groupSize) {
5     int n = hand.size();
6     if (n % groupSize != 0) return false;
7
8     unordered_map<int, int> count;
9     for (int x : hand) count[x]++;
10
11     priority_queue<int, vector<int>, greater<int>> minHeap;
12     for (auto& [val, c] : count) minHeap.push(val);
13
14     while (!minHeap.empty()) {
15         while (!minHeap.empty() && count[minHeap.top()] == 0) {
16             minHeap.pop();
17         }
18         if (minHeap.empty()) break;
19
20         int v = minHeap.top();
21         for (int j = 0; j < groupSize; j++) {
22             if (count[v + j] == 0) return false;
23             count[v + j]--;
24         }
25     }
26     return true;
27 }
28
29 int main() {
30     vector<int> hand = {1, 2, 3, 6, 2, 3, 4, 7, 8};
31     cout << (isNStraightHand(hand, 3) ? "true" : "false") << endl;
32     return 0;
33 }

```

Complexity Analysis

Time Complexity. Building the frequency map and heap costs $O(n \log n)$ in the worst case (up to n distinct values, each pushed once). The main loop performs at most $n/\text{groupSize}$ group formations, each doing $O(\text{groupSize})$ work plus $O(\log n)$ amortized heap-popping of exhausted values, giving an overall time complexity of

$$O(n \log n).$$

Space Complexity. $O(n)$ for the frequency map and heap in the worst case (all distinct values).

Key Takeaway

When a greedy choice is not just *good* but *forced* (the smallest remaining element has no alternative role to play), a min-heap is the natural data structure to repeatedly surface that forced choice efficiently, especially when – unlike a single sorted pass – elements get consumed and removed from consideration in a pattern that does not simply move left to right.

1.8 Merge K Sorted Lists

Problem Statement. We are given k linked lists, each individually sorted in increasing order. Merge all of them into a single sorted linked list and return it.

Pattern Recognition

“Merge k sorted ...” is one of the cleanest heap triggers in this chapter: merging *two* sorted lists is a familiar $O(n)$ two-pointer operation, but merging k of them at once naturally wants a data structure that can always hand you the smallest *current head* among all k lists in $O(\log k)$ time – exactly a min-heap holding one candidate node from each list.

Understanding the problem carefully

At any point during the merge, the next node to append to the output is the smallest value among the *current heads* of the k lists that still have remaining nodes (every list is individually sorted, so only its current head – not any later node – can possibly be the next-smallest value globally). A naive approach would scan all k current heads every time to find the minimum, costing $O(k)$ per output node and $O(nk)$ overall (where n is the total number of nodes). A min-heap reduces the “find the minimum among k candidates” step to $O(\log k)$.

Greedy Choice, Powered by a Min-Heap

Initialize a min-heap with the head node of each of the k lists (if non-null). Repeatedly: pop the smallest node from the heap, append it to the output list, and if that node has a successor within its own original list, push that successor onto the heap. Continue until the heap is empty.

Why popping the heap’s minimum is always the correct next output node

The heap, at every step, holds exactly one “current head” candidate per list that still has unconsumed nodes – and because each list is individually sorted, the true global minimum of all *remaining* nodes across all lists must be one of these k current heads (a node deeper inside a list cannot be smaller than that same list’s own current head). So the heap’s minimum at any moment is guaranteed to be the correct next node for the merged output, and pushing that node’s successor immediately after popping it maintains the invariant that the heap always holds the current head of every still-nonempty list.

Algorithm

1. Initialize a min-heap (ordered by node value) with the head of each of the k lists, skipping any empty lists.
2. Initialize an empty output list (with a dummy head for easy appending).

3. While the heap is not empty:
 - (a) Pop the node with the smallest value; append it to the output list.
 - (b) If this node has a **next** pointer (within its original list): push that next node onto the heap.
4. Return the output list (skipping the dummy head).

Algorithm 7 Merge K Sorted Lists

```

1: procedure MERGEKLISTS(lists)
2:   heap  $\leftarrow$  empty min-heap (by node value)
3:   for each list L in lists do
4:     if L is not empty then push L's head onto heap
5:     end if
6:   end for
7:   dummy  $\leftarrow$  new empty node; tail  $\leftarrow$  dummy
8:   while heap is not empty do
9:     node  $\leftarrow$  pop minimum from heap
10:    tail.next  $\leftarrow$  node; tail  $\leftarrow$  node
11:    if node.next  $\neq$  null then
12:      push node.next onto heap
13:    end if
14:  end while
15:  return dummy.next
16: end procedure

```

Dry run

Dry Run. Take three lists: $L_1 = 1 \rightarrow 4 \rightarrow 5$, $L_2 = 1 \rightarrow 3 \rightarrow 4$, $L_3 = 2 \rightarrow 6$.

- Initialize heap with the three heads: $\{1_{(L_1)}, 1_{(L_2)}, 2_{(L_3)}\}$.
- Pop a 1 (say from L_1): output = 1. Push L_1 's next, 4. Heap = $\{1_{(L_2)}, 2_{(L_3)}, 4_{(L_1)}\}$.
- Pop 1 (from L_2): output = 1, 1. Push L_2 's next, 3. Heap = $\{2_{(L_3)}, 3_{(L_2)}, 4_{(L_1)}\}$.
- Pop 2 (from L_3): output = 1, 1, 2. Push L_3 's next, 6. Heap = $\{3_{(L_2)}, 4_{(L_1)}, 6_{(L_3)}\}$.
- Pop 3 (from L_2): output = 1, 1, 2, 3. Push L_2 's next, 4. Heap = $\{4_{(L_1)}, 4_{(L_2)}, 6_{(L_3)}\}$.
- Pop a 4 (say from L_1): output = 1, 1, 2, 3, 4. Push L_1 's next, 5. Heap = $\{4_{(L_2)}, 5_{(L_1)}, 6_{(L_3)}\}$.
- Pop 4 (from L_2): output = 1, 1, 2, 3, 4, 4. L_2 exhausted, push nothing. Heap = $\{5_{(L_1)}, 6_{(L_3)}\}$.
- Pop 5: output = ..., 5. L_1 exhausted. Heap = $\{6_{(L_3)}\}$.
- Pop 6: output = ..., 6. Heap empty.

Final merged list: 1, 1, 2, 3, 4, 4, 5, 6.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct ListNode {
5      int val;
6      ListNode* next;
7      ListNode(int v) : val(v), next(nullptr) {}
8  };
9
10 struct Compare {
11     bool operator()(ListNode* a, ListNode* b) {
12         return a->val > b->val; // min-heap by value
13     }
14 };
15
16 ListNode* mergeKLists(vector<ListNode*>& lists) {
17     priority_queue<ListNode*, vector<ListNode*>, Compare> minHeap;
18
19     for (ListNode* node : lists) {
20         if (node) minHeap.push(node);
21     }
22
23     ListNode dummy(0);
24     ListNode* tail = &dummy;
25
26     while (!minHeap.empty()) {
27         ListNode* node = minHeap.top();
28         minHeap.pop();
29
30         tail->next = node;
31         tail = node;
32
33         if (node->next) minHeap.push(node->next);
34     }
35     return dummy.next;
36 }
37
38 int main() {
39     ListNode* l1 = new ListNode(1);
40     l1->next = new ListNode(4);
41     l1->next->next = new ListNode(5);
42
43     ListNode* l2 = new ListNode(1);
44     l2->next = new ListNode(3);
45     l2->next->next = new ListNode(4);
46
47     ListNode* l3 = new ListNode(2);
48     l3->next = new ListNode(6);
49
50     vector<ListNode*> lists = {l1, l2, l3};
51     ListNode* merged = mergeKLists(lists);
52
53     for (ListNode* cur = merged; cur; cur = cur->next) {
54         cout << cur->val << " ";
55     }
56     cout << endl;

```

```

57     return 0;
58 }

```

Complexity Analysis

Time Complexity. The heap never holds more than k nodes. Every one of the n total nodes (across all k lists) is pushed once and popped once, each costing $O(\log k)$, giving an overall time complexity of

$$O(n \log k).$$

Space Complexity. $O(k)$ for the heap (the output list reuses the input nodes rather than allocating new ones).

Key Takeaway

“Merge k sources, each individually sorted” always reduces to “track one current candidate per source in a min-heap, pop the global minimum, advance that source.” This template applies identically whether the sources are linked lists (this section), arrays, or streams, and is one of the most directly reusable patterns in this chapter.

1.9 Replace Elements by Rank in the Array

Problem Statement. Given an array *arr* of n integers, replace every element with its **rank** when the array’s distinct values are sorted in increasing order: the smallest value gets rank 1, the next distinct value gets rank 2, and so on; equal elements receive the same rank.

Pattern Recognition

“Replace each element by its rank in sorted order” is, at its core, a sorting problem – but presenting it via a min-heap makes the **“process elements in increasing order while remembering where each one came from”** pattern explicit, which is the same underlying idea used throughout this chapter whenever output order must differ from input order. Whenever you need to process values in sorted order *and* write results back to their original positions, pair each value with its original index before pushing it into the heap.

Understanding the problem carefully

Rank assignment requires processing values from smallest to largest while remembering, for each value, which original array position it came from (so the computed rank can be written back to the correct place). A min-heap of $(value, originalIndex)$ pairs, ordered by value, gives exactly this controlled smallest-to-largest traversal.

Greedy Choice, Powered by a Min-Heap

Push every $(value, originalIndex)$ pair into a min-heap ordered by value. Maintain a running rank counter, starting at 0, and a record of the previously popped value (initially undefined). Repeatedly pop the smallest remaining pair: if its value differs from the previously popped value (or this is the first pop), increment the rank counter; write the current rank counter’s value into the result array at the popped pair’s original index.

Why incrementing the rank only on a value change is correct

Equal elements must receive the same rank, and the rank must strictly increase only when a genuinely new, larger distinct value is encountered. Since the heap pops values in non-

decreasing order, comparing each popped value to the immediately previous one correctly detects exactly the moments where a new distinct value begins – there is no need to pre-deduplicate the array, because consecutive equal pops are detected and handled identically (no rank increment) regardless of how many times a value repeats.

Algorithm

1. Push every $(arr[i], i)$ pair onto a min-heap, ordered by value.
2. Initialize $rank \leftarrow 0$, $prevValue \leftarrow \text{undefined}$.
3. While the heap is not empty:
 - (a) Pop $(value, idx)$.
 - (b) If $value \neq prevValue$ (or this is the first pop): $rank \leftarrow rank + 1$; $prevValue \leftarrow value$.
 - (c) $result[idx] \leftarrow rank$.
4. Return $result$.

Algorithm 8 Replace Elements by Rank

```

1: procedure REPLACEBYRANK( $arr$ )
2:    $heap \leftarrow$  min-heap of  $(arr[i], i)$  for all  $i$ 
3:    $rank \leftarrow 0$ ,  $prevValue \leftarrow \text{undefined}$ 
4:   while  $heap$  is not empty do
5:      $(value, idx) \leftarrow$  pop minimum
6:     if  $value \neq prevValue$  then
7:        $rank \leftarrow rank + 1$ 
8:        $prevValue \leftarrow value$ 
9:     end if
10:     $result[idx] \leftarrow rank$ 
11:  end while
12:  return  $result$ 
13: end procedure

```

Dry run

Dry Run. Take $arr = [20, 15, 26, 2, 98, 6]$.

Heap pairs: $(20, 0), (15, 1), (26, 2), (2, 3), (98, 4), (6, 5)$.

- Pop $(2, 3)$: new value. $rank = 1$. $result[3] = 1$.
- Pop $(6, 5)$: new value. $rank = 2$. $result[5] = 2$.
- Pop $(15, 1)$: new value. $rank = 3$. $result[1] = 3$.
- Pop $(20, 0)$: new value. $rank = 4$. $result[0] = 4$.
- Pop $(26, 2)$: new value. $rank = 5$. $result[2] = 5$.
- Pop $(98, 4)$: new value. $rank = 6$. $result[4] = 6$.

Final result: $[4, 3, 5, 1, 6, 2]$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> replaceByRank(vector<int>& arr) {
5     int n = arr.size();
6     priority_queue<pair<int,int>, vector<pair<int,int>>,
7         greater<pair<int,int>>> minHeap;
8
9     for (int i = 0; i < n; i++) {
10         minHeap.push({arr[i], i});
11     }
12
13     vector<int> result(n);
14     int rank = 0;
15     bool first = true;
16     int prevValue = -1;
17
18     while (!minHeap.empty()) {
19         auto [value, idx] = minHeap.top();
20         minHeap.pop();
21
22         if (first || value != prevValue) {
23             rank++;
24             prevValue = value;
25             first = false;
26         }
27         result[idx] = rank;
28     }
29     return result;
30 }
31
32 int main() {
33     vector<int> arr = {20, 15, 26, 2, 98, 6};
34     vector<int> result = replaceByRank(arr);
35
36     for (int x : result) cout << x << " ";
37     cout << endl;
38     return 0;
39 }

```

Complexity Analysis

Time Complexity. Pushing n pairs costs $O(n \log n)$, and popping all n pairs costs another $O(n \log n)$, giving an overall time complexity of

$$O(n \log n).$$

Space Complexity. $O(n)$ for the heap and the result array.

Key Takeaway

Whenever a problem needs values processed in sorted order while results must land back at their *original* positions, push $(value, originalIndex)$ pairs together, rather than sorting the values alone and losing track of where each one belongs. This small habit – carry the index along with the value – recurs constantly whenever sorting (or heap order) and output

position diverge.

1.10 Task Scheduler

Problem Statement. We are given an array *tasks*, where each element is a task type (e.g. a character), and an integer *n*, the **cooldown**: the same task type cannot be executed again until at least *n* units of time have passed since its last execution (the CPU may sit **idle** for a unit of time if no eligible task is available). Find the **minimum total time** needed to execute all tasks.

Pattern Recognition

“Always schedule whichever task type currently has the most remaining occurrences (subject to a cooldown), repeatedly” is a perpetually-resorted greedy priority – exactly the *“schedule/group with a cost or cooldown that always prefers the largest remaining count first”* keyword pattern from Section 1.1. Scheduling the most frequent task type first, whenever it is eligible, spreads out the most demanding task type as early and as evenly as possible, minimizing the idle time it would otherwise force later.

Understanding the problem carefully

Intuitively, the task type with the highest remaining count is the most constraining: if we delay it, we are more likely to be forced into idle slots later, because its cooldown keeps recurring. So at every time step, we should greedily execute whichever *eligible* task type currently has the largest remaining count. A max-heap keyed by remaining count gives us this “most frequent remaining task” in $O(\log d)$ time, where *d* is the number of distinct task types (at most 26 for lowercase letters, so effectively a small constant).

Greedy Choice, Powered by a Max-Heap

Count the frequency of each task type and push these counts into a max-heap. Process time in chunks of $n + 1$ consecutive units (one full cooldown cycle): within each chunk, repeatedly pop the most frequent remaining task type, decrement its count, and execute it in the next time slot; if its count is still positive after decrementing, set it aside (it cannot be reused within this same cooldown cycle) to be pushed back into the heap only after the chunk completes. If both the heap and the set-aside list are empty before filling all $n + 1$ slots of the current chunk, stop immediately (no trailing idle time is needed); otherwise, any unfilled slots in the chunk count as idle time.

Why processing in cooldown-sized chunks is correct

Within a single chunk of $n + 1$ consecutive time slots, no task type can appear twice (that is exactly what the cooldown *n* forbids – a task executed at the start of a chunk cannot be executed again until at least *n* slots later, which is precisely the boundary of the next chunk). So a chunk can hold at most one execution of each distinct task type, and greedily filling each chunk with the currently most-frequent remaining task types (up to $\min(d, n + 1)$ of them, where *d* is how many distinct types still have remaining work) never wastes an opportunity: any slot left empty in a chunk after all distinct remaining types have been used *must* be idle, because no task is eligible. Choosing the *most frequent* remaining types to fill each chunk (rather than any other selection) is what minimizes the number of future chunks that will be forced to leave slots idle, since it depletes the most constraining task types fastest, evening out the workload as quickly as possible.

Algorithm

1. Count the frequency of each task type; push all counts into a max-heap.
2. Initialize $time \leftarrow 0$.
3. While the heap is not empty:
 - (a) $tempList \leftarrow$ empty list (tasks used this cycle, temporarily held out of the heap).
 - (b) For $i \leftarrow 0$ to n ($n + 1$ slots in this cycle):
 - i. If the heap is not empty: pop the largest count, decrement it by 1; if still > 0 , append it to $tempList$.
 - ii. $time \leftarrow time + 1$.
 - iii. If the heap is empty and $tempList$ is empty: break out of this inner loop early (no trailing idle needed).
 - (c) Push every count in $tempList$ back onto the heap.
4. Return $time$.

Algorithm 9 Task Scheduler

```

1: procedure LEASTINTERVAL( $tasks, n$ )
2:   count frequencies; push all into max-heap  $heap$ 
3:    $time \leftarrow 0$ 
4:   while  $heap$  is not empty do
5:      $temp \leftarrow$  empty list
6:     for  $i \leftarrow 0$  to  $n$  do
7:       if  $heap$  is not empty then
8:          $c \leftarrow$  pop maximum;  $c \leftarrow c - 1$ 
9:         if  $c > 0$  then append  $c$  to  $temp$ 
10:        end if
11:      end if
12:       $time \leftarrow time + 1$ 
13:      if  $heap$  is empty and  $temp$  is empty then
14:        break
15:      end if
16:    end for
17:    push every count in  $temp$  back onto  $heap$ 
18:  end while
19:  return  $time$ 
20: end procedure

```

Dry run

Dry Run. Take $tasks = [A, A, A, B, B, B]$, $n = 2$.

Counts: $A:3, B:3$. Heap = $\{3_A, 3_B\}$.

- **Cycle 1** (3 slots, since $n + 1 = 3$): pop A (or B ; ties broken arbitrarily), decrement to 2, set aside. Slot used, $time = 1$. Pop B , decrement to 2, set aside. Slot used, $time = 2$. Heap now empty, temp has $\{2_A, 2_B\}$ but temp is not empty, so we still fill the third slot: heap is empty, so this slot is idle. $time = 3$. Push $2_A, 2_B$ back: heap = $\{2_A, 2_B\}$.
- **Cycle 2:** pop A (decrement to 1, set aside), $time = 4$. Pop B (decrement to 1, set aside), $time = 5$. Third slot: heap empty, temp not empty, idle. $time = 6$. Push $1_A, 1_B$ back.

- **Cycle 3:** pop A (decrement to 0, do not set aside), $time = 7$. Pop B (decrement to 0, do not set aside), $time = 8$. Now heap is empty and temp is empty: break early, no third idle slot needed.

Total time is 8, matching the well-known schedule $A, B, \text{idle}, A, B, \text{idle}, A, B$.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int leastInterval(vector<char>& tasks, int n) {
5     unordered_map<char,int> freq;
6     for (char t : tasks) freq[t]++;
7
8     priority_queue<int> maxHeap;
9     for (auto& [ch, c] : freq) maxHeap.push(c);
10
11     int time = 0;
12     while (!maxHeap.empty()) {
13         vector<int> temp;
14         int slots = n + 1;
15
16         for (int i = 0; i < slots; i++) {
17             if (!maxHeap.empty()) {
18                 int c = maxHeap.top() - 1;
19                 maxHeap.pop();
20                 if (c > 0) temp.push_back(c);
21             }
22             time++;
23             if (maxHeap.empty() && temp.empty()) break;
24         }
25         for (int c : temp) maxHeap.push(c);
26     }
27     return time;
28 }
29
30 int main() {
31     vector<char> tasks = {'A','A','A','B','B','B'};
32     cout << "Minimum time: " << leastInterval(tasks, 2) << endl;
33     return 0;
34 }

```

Complexity Analysis

Time Complexity. There are at most $d \leq 26$ distinct task types, so the heap has size $O(1)$ from a complexity standpoint, and the total number of pop/push operations is bounded by the total task count N (each task is popped and re-pushed $O(1)$ times across all cycles in the typical analysis), giving

$$O(N \log d) = O(N),$$

effectively linear in the number of tasks. **Space Complexity.** $O(d)$ for the frequency map and heap.

Key Takeaway

A closed-form formula also exists for this specific problem ($answer = \max(N, (maxFreq - 1)(n + 1) + numTasksWithMaxFreq)$), but the heap-based simulation shown here generalizes better: it directly extends to variants with per-task-type costs, partial cooldowns, or other constraints where a clean closed form may not exist, while the underlying “always schedule the currently most-constraining option” greedy idea remains unchanged.

1.11 Design Twitter

Problem Statement. Design a simplified Twitter supporting: *postTweet(userId, tweetId)* – a user composes a new tweet; *getNewsFeed(userId)* – return the 10 most recent tweet IDs from the given user and everyone they follow, most recent first; *follow(followerId, followeeId)*; and *unfollow(followerId, followeeId)*.

Pattern Recognition

“Most recent k items, drawn from several independently time-ordered sources” is exactly Merge K Sorted Lists (Section 1.8) combined with the Kth Largest size-bounded heap idea (Section 1.5), applied together: each followed user’s tweet history is individually time-ordered (most recent last, or equivalently, sorted by a timestamp), and we need the top 10 across all of them – a direct fusion of two patterns already established in this chapter, now applied inside a larger system-design problem.

Understanding the problem carefully

Each user’s tweets, in posting order, already form an individually sorted sequence (sorted by time posted). Getting a news feed means merging the relevant followed users’ tweet sequences and taking the top 10 most recent – structurally identical to merging k sorted lists and only needing the first 10 elements of the merge, rather than the full merge. We assign a global, strictly increasing *timestamp* to every tweet at the moment it is posted (a simple counter, incremented on every *postTweet* call across *all* users), so that “most recent” always corresponds to “largest timestamp,” comparable across different users.

Data Structures

- A global counter *timestamp*, incremented on every post.
- For each user, a list of $(timestamp, tweetId)$ pairs, in posting order (so the most recent tweet is always at the end).
- For each user, a set of followee user IDs (a user is always considered to follow themselves, for news-feed purposes, without needing an explicit *follow* call).

News Feed via a Bounded Max-Heap

For *getNewsFeed(userId)*: for each followee (including the user themselves), look only at their *most recent* tweet first (the tail of their tweet list) – exactly like the “current head of each sorted source” idea from merging k sorted lists, just read from the end instead of the front, since within a user’s own list, later postings have larger timestamps. Push each followee’s most recent tweet into a max-heap keyed by timestamp. Repeatedly pop the global maximum, record its tweet ID, and if that same followee has an earlier tweet remaining (walking backward through their list), push that earlier tweet in its place. Stop

after popping 10 tweets or exhausting the heap.

Why this bounded heap pop is correct and efficient

This is precisely the merge- k -sorted-lists technique, with two adjustments: we traverse each followee's list from most recent to least recent (so the heap always offers the next-best candidate, by timestamp, in the right direction), and we stop after 10 pops rather than draining the entire merge, since the problem only ever wants the top 10. Stopping early is valid precisely because a max-heap's repeated-pop sequence *is* a fully sorted (by timestamp, descending) traversal of the union of all followees' tweets – truncating that sorted traversal at 10 elements is exactly the 10 most recent tweets, with no need to ever materialize the full merge.

Algorithm

1. POSTTWEET(*userId*, *tweetId*): append (*timestamp*++, *tweetId*) to *userId*'s tweet list.
2. FOLLOW/UNFOLLOW: add/remove *followeeId* from *followerId*'s followee set.
3. GETNEWSFEED(*userId*):
 - (a) For each followee f in *userId*'s followee set (plus *userId* itself): if f has at least one tweet, push (*their last tweet's timestamp*, f , *index of that tweet*) onto a max-heap.
 - (b) Repeat up to 10 times: pop the maximum; record its tweet ID; if the popped followee has an earlier tweet at $index - 1$, push that one.
 - (c) Return the recorded tweet IDs, in the order popped (already most-recent-first).

Dry run

Dry Run. User 1 posts tweets 5 (at timestamp 1) and 3 (at timestamp 2). User 2 posts tweet 101 (at timestamp 3). User 1 follows User 2. GETNEWSFEED(1) is called.

User 1's followees (including self): {1, 2}. Most recent tweet of user 1: (2, 3) – timestamp 2, tweet 3. Most recent tweet of user 2: (3, 101) – timestamp 3, tweet 101. Heap initialized with both.

- Pop max: (3, 101) from user 2. Record tweet 101. User 2 has no earlier tweet; nothing pushed back.
- Pop max: (2, 3) from user 1. Record tweet 3. User 1 has an earlier tweet at timestamp 1 (tweet 5); push (1, 5).
- Pop max: (1, 5) from user 1. Record tweet 5. No earlier tweet from user 1.
- Heap empty; fewer than 10 tweets total, so we stop here anyway.

The news feed is [101, 3, 5], most recent first – correct.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class Twitter {
5     int timestamp = 0;

```

```

6      unordered_map<int, vector<pair<int,int>>> tweets; // userId -> (
           time, tweetId)
7      unordered_map<int, unordered_set<int>> following; // userId ->
           followees
8
9  public:
10     void postTweet(int userId, int tweetId) {
11         tweets[userId].push_back({timestamp++, tweetId});
12     }
13
14     vector<int> getNewsFeed(int userId) {
15         // (timestamp, userId, index into that user's tweet list)
16         priority_queue<tuple<int,int,int>> maxHeap;
17
18         auto consider = [&](int uid) {
19             if (!tweets[uid].empty()) {
20                 int idx = tweets[uid].size() - 1;
21                 maxHeap.push({tweets[uid][idx].first, uid, idx});
22             }
23         };
24
25         consider(userId);
26         for (int f : following[userId]) consider(f);
27
28         vector<int> result;
29         while (!maxHeap.empty() && (int)result.size() < 10) {
30             auto [time, uid, idx] = maxHeap.top();
31             maxHeap.pop();
32             result.push_back(tweets[uid][idx].second);
33
34             if (idx > 0) {
35                 maxHeap.push({tweets[uid][idx-1].first, uid, idx-1});
36             }
37         }
38         return result;
39     }
40
41     void follow(int followerId, int followeeId) {
42         if (followerId != followeeId) following[followerId].insert(
43             followeeId);
44     }
45
46     void unfollow(int followerId, int followeeId) {
47         following[followerId].erase(followeeId);
48     }
49
50     int main() {
51         Twitter tw;
52         tw.postTweet(1, 5);
53         tw.postTweet(1, 3);
54         tw.postTweet(2, 101);
55         tw.follow(1, 2);
56
57         for (int id : tw.getNewsFeed(1)) cout << id << " ";
58         cout << endl;
59         return 0;
60     }

```

Complexity Analysis

Time Complexity. *postTweet*, *follow*, and *unfollow* are $O(1)$ amortized. *getNewsFeed* pushes at most f initial candidates (one per followee, $f \leq$ number followed) and performs at most 10 pop/push rounds, each $O(\log f)$, giving

$$O(f \log f)$$

per call, independent of the total number of tweets ever posted. **Space Complexity.** $O(T)$ to store all T tweets ever posted, plus $O(f)$ per news feed query for the heap.

Key Takeaway

System-design problems frequently turn out to be direct compositions of patterns you already know: this one is exactly “merge k sorted sources” (Section 1.8) plus “stop after the top k ” (Section 1.5), wrapped in bookkeeping for followers and a global timestamp. Recognizing the composition is more valuable than treating it as an unfamiliar new problem.

1.12 Find Median from a Data Stream

Problem Statement. Design a structure supporting *addNum*(x), which adds an integer to a growing stream, and *findMedian*(), which returns the median of all numbers added so far, at any point, efficiently after every insertion.

Pattern Recognition

“Running median, computed after every new element of a stream” is the canonical trigger for the **two-heap** pattern flagged in Section 1.1: a median needs access to both “the largest of the lower half” and “the smallest of the upper half” of the data seen so far, and no single heap can offer both – so we keep two heaps of opposite polarity, each covering one half, and keep them balanced.

Understanding the problem carefully

If we maintain the invariant that all elements in a **max-heap** called *lower* are $\leq$ all elements in a **min-heap** called *upper*, and that the two heaps’ sizes differ by at most 1, then:

- If the total count is odd, the median is the top of whichever heap has one more element than the other.
- If the total count is even, the median is the average of *lower*’s top (the largest of the lower half) and *upper*’s top (the smallest of the upper half).

The entire problem reduces to maintaining this “balanced split into a max-heap lower half and a min-heap upper half” invariant after every insertion.

Insertion Rule

On *addNum*(x): push x into *lower* (the max-heap). Then, to preserve the “everything in *lower* is $\leq$ everything in *upper*” invariant, pop *lower*’s maximum and push it into *upper* (this guarantees the largest element currently in *lower* is never larger than anything already confirmed to belong in the upper half). Finally, to preserve the size-balance invariant: if *upper* now has more elements than *lower*, pop *upper*’s minimum and push it back into *lower*.

Why pushing into *lower* first, then shuffling, is correct

Always inserting x into *lower* first (regardless of whether x is actually a “low” or “high” value) and then immediately moving *lower*’s current maximum into *upper* is a clean way to guarantee correctness without an explicit case split on where x belongs: after this shuffle, *upper*’s new minimum is exactly $\min(x, \text{old } upper\text{'s minimum})$, and one can verify that whichever of these is smaller is correctly placed at the boundary, while *lower*’s new maximum is whatever was second-largest among $\{x\} \cup \text{lower's old elements}$, preserving $lower \leq upper$ throughout. The subsequent size-rebalancing step (moving one element back if *upper* becomes too large) maintains the size-difference-at-most-1 invariant, which is exactly what makes the parity-based median formula above always correct.

Algorithm

1. ADDNUM(x):
 - (a) Push x onto *lower* (max-heap).
 - (b) Pop *lower*’s max; push it onto *upper* (min-heap).
 - (c) If $|upper| > |lower|$: pop *upper*’s min; push it back onto *lower*.
2. FINDMEDIAN():
 - (a) If $|lower| > |upper|$: return *lower*’s top.
 - (b) Else: return (*lower*’s top + *upper*’s top)/2.

Algorithm 10 Median Maintainer (Two Heaps)

```

1: procedure ADDNUM( $x$ )
2:   push  $x$  onto lower (max-heap)
3:   push (pop lower) onto upper (min-heap)
4:   if  $|upper| > |lower|$  then
5:     push (pop upper) onto lower
6:   end if
7: end procedure
8: procedure FINDMEDIAN
9:   if  $|lower| > |upper|$  then return top of lower
10:  elsereturn (top of lower + top of upper)/2
11:  end if
12: end procedure

```

Dry run

Dry Run. Add 5, 15, 1, 3 in order, checking the median after each.

- Add 5: push to *lower* = {5}. Pop max (5), push to *upper* = {5}. Now $|upper| = 1 > |lower| = 0$: pop *upper*’s min (5), push to *lower*. *lower* = {5}, *upper* = {}. Median: $|lower| > |upper|$, so median = 5.
- Add 15: push to *lower* = {5, 15} (max-heap top 15). Pop max (15), push to *upper* = {15}. $|upper| = 1 = |lower| = 1$, no rebalance needed. Median: equal sizes, $(5 + 15)/2 = 10$.
- Add 1: push to *lower* = {1, 5} (top 5). Pop max (5), push to *upper* = {5, 15} (top 5). $|upper| = 2 > |lower| = 1$: pop *upper*’s min (5), push to *lower* = {1, 5} (top 5). *upper* = {15}. Median: $|lower| = 2 > |upper| = 1$, median = 5.

- Add 3: push to $lower = \{1, 3, 5\}$ (top 5). Pop max (5), push to $upper = \{5, 15\}$ (top 5). $|upper| = 2 = |lower| = 2$, no rebalance. Median: equal sizes, (top of lower = 3 + top of upper = 5)/2 = 4.

Checking by hand: after $[5, 15, 1, 3]$, sorted is $[1, 3, 5, 15]$, median = $(3 + 5)/2 = 4$. Correct.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 class MedianFinder {
5     priority_queue<int> lower; // max-heap: lower half
6     priority_queue<int, vector<int>, greater<int>> upper; // min-heap:
        upper half
7
8 public:
9     void addNum(int x) {
10         lower.push(x);
11         upper.push(lower.top());
12         lower.pop();
13
14         if (upper.size() > lower.size()) {
15             lower.push(upper.top());
16             upper.pop();
17         }
18     }
19
20     double findMedian() {
21         if (lower.size() > upper.size()) {
22             return lower.top();
23         }
24         return (lower.top() + upper.top()) / 2.0;
25     }
26 };
27
28 int main() {
29     MedianFinder mf;
30     for (int x : {5, 15, 1, 3}) {
31         mf.addNum(x);
32         cout << "Median after adding " << x << ": "
33              << mf.findMedian() << endl;
34     }
35     return 0;
36 }

```

Complexity Analysis

Time Complexity. Each *addNum* performs a constant number of heap pushes and pops, each $O(\log n)$, giving

$$O(\log n) \text{ per insertion,} \quad O(1) \text{ per median query.}$$

Space Complexity. $O(n)$ to store all elements seen so far, split between the two heaps.

Key Takeaway

The two-heap pattern – a max-heap for “everything below the median” and a min-heap for “everything above,” kept balanced in size – is the standard way to maintain order statistics (median, or more generally any fixed percentile with a fixed split ratio) over a stream that only grows. Whenever a running statistic needs comparison to *both* a lower and an upper neighborhood simultaneously, suspect this pattern before reaching for a single heap.

1.13 Kth Largest Element in a Stream

Problem Statement. Design a class initialized with an integer k and an initial array $nums$, supporting $add(val)$: add val to the running stream and return the k -th largest element among **all** elements seen so far (including the initial array and every value added).

Pattern Recognition

This is Section 1.5’s “Kth Largest Element in an Array” made **incremental**: instead of computing the answer once for a fixed array, we must maintain it continuously as new elements stream in, one at a time. The keyword shift from a single static query to a class with a repeated add method is the signal that the size- k min-heap should be built *once* and *kept alive* across calls, rather than recomputed from scratch on every query.

Understanding the problem carefully

The min-heap-of-size- k technique from Section 1.5 already processes elements one at a time and remains correct at *every intermediate point* during that processing – the heap, after processing any prefix of elements, always holds the k largest elements seen *so far*, with the k -th largest sitting at the root. This means no new derivation is needed: we simply keep the heap object alive as part of the class’s state, seed it with the initial array, and apply the exact same push-then-conditionally-pop step to each newly added value.

Reuse: Make the Heap Persistent

On construction: build a min-heap from the initial $nums$, trimming it down to size k using the same push-then-pop-if-oversized rule as Algorithm 1.5. On each $add(val)$: apply that same rule to val alone, then return the heap’s current root.

Algorithm

1. **CONSTRUCTOR**($k, nums$): for each x in $nums$, push x onto the min-heap; if size exceeds k , pop the minimum.
2. **ADD**(val): push val onto the min-heap; if size exceeds k , pop the minimum; return the heap’s current top.

Algorithm 11 Kth Largest Element in a Stream

```

1: procedure CONSTRUCTOR( $k$ ,  $nums$ )
2:    $heap \leftarrow$  empty min-heap
3:   for each  $x$  in  $nums$  do
4:     push  $x$  onto  $heap$ 
5:     if  $|heap| > k$  then pop the minimum
6:   end if
7: end for
8: end procedure
9: procedure ADD( $val$ )
10:  push  $val$  onto  $heap$ 
11:  if  $|heap| > k$  then pop the minimum
12:  end if
13:  return top of  $heap$ 
14: end procedure

```

Dry run

Dry Run. Take $k = 3$, initial $nums = [4, 5, 8, 2]$, then call $add(3)$, $add(5)$, $add(10)$.

- Construct: push 4, 5, 8, 2 one at a time, trimming to size 3 each time it exceeds 3. After all four: heap holds the 3 largest of $\{4, 5, 8, 2\}$, which are $\{4, 5, 8\}$ (root = 4).
- $add(3)$: push 3; heap = $\{3, 4, 5, 8\}$, size 4 > 3: pop min (3). Heap = $\{4, 5, 8\}$. Return 4.
- $add(5)$: push 5; heap = $\{4, 5, 5, 8\}$, size 4 > 3: pop min (4). Heap = $\{5, 5, 8\}$. Return 5.
- $add(10)$: push 10; heap = $\{5, 5, 8, 10\}$, size 4 > 3: pop min (5). Heap = $\{5, 8, 10\}$. Return 5.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class KthLargest {
5      int k;
6      priority_queue<int, vector<int>, greater<int>> minHeap;
7
8  public:
9      KthLargest(int k, vector<int>& nums) : k(k) {
10         for (int x : nums) {
11             minHeap.push(x);
12             if ((int)minHeap.size() > k) minHeap.pop();
13         }
14     }
15
16     int add(int val) {
17         minHeap.push(val);
18         if ((int)minHeap.size() > k) minHeap.pop();
19         return minHeap.top();
20     }
21 };
22
23 int main() {
24     vector<int> nums = {4, 5, 8, 2};

```

```

25     KthLargest kl(3, nums);
26
27     cout << kl.add(3) << endl;
28     cout << kl.add(5) << endl;
29     cout << kl.add(10) << endl;
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. Construction costs $O(n \log k)$ for the initial n elements. Each subsequent *add* call costs $O(\log k)$. **Space Complexity.** $O(k)$, since the heap never exceeds size k .

Key Takeaway

Many “streaming” versions of array problems require no new algorithmic idea at all – only recognizing that the data structure from the static version (here, a size- k min-heap) is already incremental by construction, and simply needs to be kept alive as persistent state rather than rebuilt on every query.

1.14 Maximum Sum Combination

Problem Statement. We are given two arrays A and B , each of size N , and an integer K . Consider every possible pair sum $A[i] + B[j]$ for $0 \leq i, j < N$ (there are N^2 such pairs). Find the K **largest** pair sums, in descending order, **without** computing all N^2 sums.

Pattern Recognition

“ K largest sums formed by combining one element from each of two (sorted) collections” is a generalization of Kth Largest (Section 1.5) to a combinatorially large candidate space: there are N^2 possible pairs, far too many to generate and sort when $K \ll N^2$. The keyword to notice is that the candidate sums have **internal structure** (they come from a grid of sorted index pairs), which a heap can exploit to generate only the K largest sums directly, without ever materializing the rest.

Understanding the problem carefully

Sort both A and B in **descending** order first. Now, $A[0] + B[0]$ is guaranteed to be the single largest possible sum (the largest element of each array, paired together). Crucially, once we know $A[0] + B[0]$ is the maximum, the *next*-largest sum must come from either decreasing the A -index to 1 (pairing $A[1]$ with $B[0]$) or decreasing the B -index to 1 (pairing $A[0]$ with $B[1]$) – no other pair can exceed both of these, since both arrays are sorted descending, and any pair using indices further from $(0, 0)$ in either coordinate can only be smaller than at least one of these two candidates.

Greedy Choice, Powered by a Max-Heap over Index Pairs

Maintain a max-heap of candidate *index pairs* (i, j) , keyed by $A[i] + B[j]$, along with a *visited* set to avoid pushing the same pair twice. Start with $(0, 0)$ in the heap. Repeat K times: pop the pair with the largest sum, record that sum, then push $(i + 1, j)$ and $(i, j + 1)$ (if within bounds and not already visited) as the next candidates that could potentially be the next-largest sum.

Why only two neighbors need to be considered at each step

This is the same idea as merging k sorted lists (Section 1.8), generalized to a 2D grid of sorted candidates rather than k separate 1D lists: because both A and B are sorted descending, the sum $A[i] + B[j]$ is non-increasing as i or j increases (holding the other fixed) – so the grid of all possible sums is itself “sorted” along both axes, exactly like the row/column-sorted matrices in Sections ??–??. Once a pair (i, j) has been popped as the current maximum, the only pairs that could possibly be the *next*-largest, among those not yet considered, are its immediate neighbors $(i + 1, j)$ and $(i, j + 1)$ – every other still-uncovered pair is reachable only by passing through one of these two first, so they are never skipped over by this expansion rule.

Algorithm

1. Sort A and B in descending order.
2. Initialize a max-heap (by sum) with the single pair $(0, 0)$; mark $(0, 0)$ visited.
3. Repeat K times:
 - (a) Pop (i, j) with the largest sum $A[i] + B[j]$; record this sum.
 - (b) If $i + 1 < N$ and $(i + 1, j)$ not visited: push it; mark visited.
 - (c) If $j + 1 < N$ and $(i, j + 1)$ not visited: push it; mark visited.
4. Return the K recorded sums.

Algorithm 12 Maximum Sum Combination

```

1: procedure MAXSUMCOMBINATION( $A, B, K$ )
2:   sort  $A, B$  descending
3:    $heap \leftarrow$  max-heap with  $(A[0] + B[0], 0, 0)$ 
4:   mark  $(0, 0)$  visited
5:    $result \leftarrow$  empty list
6:   for  $count \leftarrow 1$  to  $K$  do
7:      $(sum, i, j) \leftarrow$  pop maximum
8:     append  $sum$  to  $result$ 
9:     if  $i + 1 < N$  and  $(i + 1, j)$  not visited then
10:      push  $(A[i + 1] + B[j], i + 1, j)$ ; mark visited
11:    end if
12:    if  $j + 1 < N$  and  $(i, j + 1)$  not visited then
13:      push  $(A[i] + B[j + 1], i, j + 1)$ ; mark visited
14:    end if
15:  end for
16:  return  $result$ 
17: end procedure

```

Dry run

Dry Run. Take $A = [3, 2]$, $B = [1, 4]$, $K = 2$. Sorted descending: $A = [3, 2]$, $B = [4, 1]$.

- Start: $heap = \{(7, 0, 0)\}$ (sum $A[0] + B[0] = 3 + 4 = 7$). Visited = $\{(0, 0)\}$.
- Pop $(7, 0, 0)$: record 7. Push $(1, 0)$: sum $A[1] + B[0] = 2 + 4 = 6$. Push $(0, 1)$: sum $A[0] + B[1] = 3 + 1 = 4$. $Heap = \{6, 4\}$. Visited = $\{(0, 0), (1, 0), (0, 1)\}$.

- Pop (6, 1, 0): record 6. $K = 2$ reached; stop.

The top 2 sums are [7, 6]. Checking by hand: all four pair sums are $3+4 = 7$, $3+1 = 4$, $2+4 = 6$, $2+1 = 3$; sorted descending, [7, 6, 4, 3], and the top 2 are indeed [7, 6].

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> maxSumCombination(vector<int> A, vector<int> B, int K) {
5     int n = A.size();
6     sort(A.rbegin(), A.rend());
7     sort(B.rbegin(), B.rend());
8
9     // (sum, i, j)
10    priority_queue<tuple<int, int, int>> maxHeap;
11    set<pair<int, int>> visited;
12
13    maxHeap.push({A[0] + B[0], 0, 0});
14    visited.insert({0, 0});
15
16    vector<int> result;
17    while (K-- > 0 && !maxHeap.empty()) {
18        auto [sum, i, j] = maxHeap.top();
19        maxHeap.pop();
20        result.push_back(sum);
21
22        if (i + 1 < n && !visited.count({i+1, j})) {
23            maxHeap.push({A[i+1] + B[j], i+1, j});
24            visited.insert({i+1, j});
25        }
26        if (j + 1 < n && !visited.count({i, j+1})) {
27            maxHeap.push({A[i] + B[j+1], i, j+1});
28            visited.insert({i, j+1});
29        }
30    }
31    return result;
32 }
33
34 int main() {
35     vector<int> A = {3, 2};
36     vector<int> B = {1, 4};
37     for (int x : maxSumCombination(A, B, 2)) cout << x << " ";
38     cout << endl;
39     return 0;
40 }

```

Complexity Analysis

Time Complexity. Sorting costs $O(N \log N)$. The main loop runs K times, each doing $O(1)$ pushes/pops of $O(\log K)$ cost (the heap never holds more than $O(K)$ candidate pairs), giving an overall time complexity of

$$O(N \log N + K \log K).$$

Space Complexity. $O(K)$ for the heap and visited set.

Key Takeaway

“ K largest/smallest results from combining elements of two or more sorted collections” generalizes the merge- k -sorted-lists idea (Section 1.8) to a combinatorial candidate space: instead of k separate sequences, the candidates form an implicit sorted grid, and a heap explores that grid one “neighbor expansion” at a time, from the best candidate outward, generating exactly the K best results without ever enumerating the rest.

1.15 Minimum Cost to Connect Sticks

Problem Statement. We are given n sticks with given lengths. We may repeatedly choose any two sticks, combining them into a single stick whose length is the sum of the two, at a **cost equal to that sum**. We repeat until only one stick remains. Find the **minimum total cost** to combine all sticks into one.

Pattern Recognition

“Repeatedly combine the two smallest remaining items, at a cost equal to their combination,” is a direct heap-driven greedy: every combination’s cost depends on the lengths being combined, and combining **small** sticks early keeps their (now-larger) sum from being re-combined and re-charged many times over as the process continues. This is structurally the same idea behind Huffman coding (building an optimal prefix-free code by always merging the two least-frequent symbols first), though no prior knowledge of Huffman coding is needed to derive it here.

Understanding the problem carefully

Every stick’s original length is ultimately added into the cost once for every combination step it participates in afterward, directly or as part of a larger merged stick. A stick combined *early* (when it is still short) participates in fewer total dollars of cost across the remaining steps than a stick combined *late* (after it has already grown large through earlier merges). So, to minimize total cost, we always want the currently smallest available sticks to be the ones combined next – deferring large sticks (or large merged sums) to participate in as few further combination steps as possible.

Greedy Choice, Powered by a Min-Heap

Push every stick length into a min-heap. Repeat until only one element remains: pop the two smallest sticks, add their sum to the running total cost, and push that sum back into the heap as a new (merged) stick.

Why always combining the two smallest sticks is optimal

This is an exchange argument identical in spirit to Huffman coding’s optimality proof: suppose an optimal combination sequence does not combine the two globally smallest sticks $x \leq y$ at some point, but instead combines one of them with some larger stick z first. One can show that swapping the order – combining x and y first instead – never increases the total cost, because x and y being the smallest means they contribute the least possible amount per combination step they are involved in, and delaying them only ever increases the number of combination steps (each adding their value again, transitively, through ever-larger merged sums) that their lengths end up contributing to. Repeatedly applying this swap argument shows that always combining the two currently smallest available sticks, at every step, is never worse than any alternative ordering.

Algorithm

1. Push every stick length into a min-heap.
2. Initialize $totalCost \leftarrow 0$.
3. While the heap has more than one element:
 - (a) Pop the two smallest values a, b .
 - (b) $totalCost \leftarrow totalCost + (a + b)$.
 - (c) Push $a + b$ back onto the heap.
4. Return $totalCost$.

Algorithm 13 Minimum Cost to Connect Sticks

```

1: procedure CONNECTSTICKS(sticks)
2:   heap  $\leftarrow$  min-heap of all stick lengths
3:   totalCost  $\leftarrow$  0
4:   while |heap| > 1 do
5:      $a \leftarrow$  pop minimum,  $b \leftarrow$  pop minimum
6:     totalCost  $\leftarrow$  totalCost +  $a + b$ 
7:     push  $a + b$  onto heap
8:   end while
9:   return totalCost
10: end procedure

```

Dry run

Dry Run. Take $sticks = [2, 4, 3]$.
 Heap = $\{2, 3, 4\}$.

- Pop 2, 3. Cost + = 5. Push 5. Heap = $\{4, 5\}$. $totalCost = 5$.
- Pop 4, 5. Cost + = 9. Push 9. Heap = $\{9\}$. $totalCost = 14$.
- Only one element remains; stop.

The minimum total cost is 14.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int connectSticks(vector<int>& sticks) {
5     priority_queue<int, vector<int>, greater<int>> minHeap(
6         sticks.begin(), sticks.end());
7
8     int totalCost = 0;
9     while (minHeap.size() > 1) {
10         int a = minHeap.top(); minHeap.pop();
11         int b = minHeap.top(); minHeap.pop();
12
13         totalCost += (a + b);

```

```

14     minHeap.push(a + b);
15 }
16 return totalCost;
17 }
18
19 int main() {
20     vector<int> sticks = {2, 4, 3};
21     cout << "Minimum cost: " << connectSticks(sticks) << endl;
22     return 0;
23 }

```

Complexity Analysis

Time Complexity. Building the initial heap costs $O(n)$ (using the bottom-up build-heap technique from Section 1.4, which most standard library implementations, including `std::priority_queue`'s range constructor, use internally). Each of the $n - 1$ combination steps performs two pops and one push, each $O(\log n)$, giving an overall time complexity of

$$O(n \log n).$$

Space Complexity. $O(n)$ for the heap.

Key Takeaway

“Repeatedly merge the two cheapest/smallest remaining items, paying a cost proportional to the merge” is the Huffman-coding greedy pattern, recognizable on sight once you have seen it: a min-heap that always surfaces the two cheapest current items, merged and pushed back, builds an optimal merge order in $O(n \log n)$ without ever needing to consider merge orders explicitly.

1.16 Top K Frequent Elements

Problem Statement. Given an array *nums* and an integer *k*, return the *k* elements that appear **most frequently** in the array.

Pattern Recognition

“Top *k* frequent” is the size-*k* heap technique from Section 1.5, applied not to the raw values themselves but to a **derived key** – here, frequency, computed once via a hash map before the heap ever comes into play. Whenever a problem asks for the top/bottom *k* items ranked by something other than their own raw value (frequency, distance, a custom score), first compute that key for every item, then apply the familiar size-*k* heap exactly as if the key were the value.

Understanding the problem carefully

First, count the frequency of every distinct element using a hash map – this converts the problem from “top *k* frequent elements” into “top *k* elements by a number we now have for each of them,” which is exactly the shape of Section 1.5, just with (*key* = *frequency*, *value* = *element*) pairs instead of raw numbers.

Reuse, with a Derived Key

Build a frequency map. Maintain a min-heap of size k , ordered by frequency, holding $(frequency, element)$ pairs. Process each distinct element: push it; if the heap exceeds size k , pop the pair with the smallest frequency (the current weakest member of the top- k -frequent set). At the end, the heap holds exactly the k most frequent elements.

Why the size- k heap argument transfers unchanged

The correctness argument from Section 1.5 never actually depended on the heap being keyed by the element's own value – it only depended on the key being a single comparable number that we want to rank by. Replacing “value” with “frequency of that value” changes nothing about why the size- k min-heap correctly converges to the top- k set: the same induction (the heap always holds the best k keys seen so far, by the same push-then-evict-the-worst logic) applies verbatim.

Algorithm

1. Build a frequency map of $nums$.
2. For each distinct $(element, freq)$ pair:
 - (a) Push $(freq, element)$ onto a min-heap (ordered by $freq$).
 - (b) If the heap's size exceeds k : pop the minimum.
3. Return the elements remaining in the heap.

Algorithm 14 Top K Frequent Elements

```

1: procedure TOPKFREQUENT( $nums, k$ )
2:   build frequency map  $freq$  of  $nums$ 
3:    $heap \leftarrow$  empty min-heap (by frequency)
4:   for each  $(elem, f)$  in  $freq$  do
5:     push  $(f, elem)$  onto  $heap$ 
6:     if  $|heap| > k$  then pop the minimum
7:   end if
8: end for
9:   return all elements remaining in  $heap$ 
10: end procedure

```

Dry run

Dry Run. Take $nums = [1, 1, 1, 2, 2, 3]$, $k = 2$.

Frequency map: 1:3, 2:2, 3:1.

- Push (3, 1): $heap = \{(3, 1)\}$.
- Push (2, 2): $heap = \{(2, 2), (3, 1)\}$. Size $2 \leq 2$.
- Push (1, 3): $heap = \{(1, 3), (2, 2), (3, 1)\}$. Size $3 > 2$: pop min (1, 3). $Heap = \{(2, 2), (3, 1)\}$.

The final heap holds elements $\{2, 1\}$, the two most frequent values (frequencies 2 and 3 respectively).

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 vector<int> topKFrequent(vector<int>& nums, int k) {
5     unordered_map<int,int> freq;
6     for (int x : nums) freq[x]++;
7
8     priority_queue<pair<int,int>, vector<pair<int,int>>,
9                 greater<pair<int,int>>> minHeap; // (freq, element)
10
11     for (auto& [elem, f] : freq) {
12         minHeap.push({f, elem});
13         if ((int)minHeap.size() > k) minHeap.pop();
14     }
15
16     vector<int> result;
17     while (!minHeap.empty()) {
18         result.push_back(minHeap.top().second);
19         minHeap.pop();
20     }
21     return result;
22 }
23
24 int main() {
25     vector<int> nums = {1, 1, 1, 2, 2, 3};
26     for (int x : topKFrequent(nums, 2)) cout << x << " ";
27     cout << endl;
28     return 0;
29 }

```

Complexity Analysis

Time Complexity. Building the frequency map costs $O(n)$. Processing each of the $d \leq n$ distinct elements costs $O(\log k)$, giving an overall time complexity of

$$O(n + d \log k).$$

Space Complexity. $O(d)$ for the frequency map plus $O(k)$ for the heap.

Key Takeaway

This closing problem of the chapter is a deliberate reminder: nearly every heap problem you will face is one of a small number of templates – size- k heap, merge- k -sources, two-balanced-heaps, or always-take-the-best-remaining-greedy – applied to whatever key the problem actually cares about ranking by. Compute that key first (here, frequency, via a hash map), and the heap logic that follows is almost always one you have already written in this chapter.